A Tour Of Sage

Release 10.10

The Sage Development Team

28 ott 2026

Indice

1 Sage come Calcolatrice **3**

2 Power Computing con Sage **7**

3 Accesso agli Algoritmi in Sage **11**

Questa e" una breve introduzione a Sage che ricalca il tour di Mathematica che si trova all'inizio del libro «The Mathematica Book»

CAPITOLO 1

Sage come Calcolatrice

La linea di comando di Sage ha un prompt `sage:`; non dovete aggiungerlo voi. Se usate il notebook di Sage, allora riportate quello che appare dopo il prompt `sage:` in una cella vuota, e premi MAIUSC+ENTER per ottenere l'output corrispondente

```
sage: 3 + 5
8
```

```
>>> from sage.all import *
>>> Integer(3) + Integer(5)
8
```

L'accento circonflesso indica «l'elevamento a potenza».

```
sage: 57.1 ^ 100
4.60904368661396e175
```

```
>>> from sage.all import *
>>> RealNumber('57.1') ** Integer(100)
4.60904368661396e175
```

Calcoliamo l'inversa di una matrice 2×2 con Sage.

```
sage: matrix([[1,2], [3,4]])^(-1)
[  -2    1]
[ 3/2 -1/2]
```

```
>>> from sage.all import *
>>> matrix([[Integer(1),Integer(2)], [Integer(3),Integer(4)]])**(-Integer(1))
[  -2    1]
[ 3/2 -1/2]
```

Qui integriamo una funzione in una variabile.

```
sage: x = var('x')   # crea una variabile simbolica
sage: integrate(sqrt(x)*sqrt(1+x), x)
1/4*((x + 1)^(3/2)/x^(3/2) + sqrt(x + 1)/sqrt(x))/((x + 1)^2/x^2 - 2*(x + 1)/x + 1) -␣
↪1/8*log(sqrt(x + 1)/sqrt(x) + 1) + 1/8*log(sqrt(x + 1)/sqrt(x) - 1)
```

```
>>> from sage.all import *
>>> x = var('x')   # crea una variabile simbolica
>>> integrate(sqrt(x)*sqrt(Integer(1)+x), x)
1/4*((x + 1)^(3/2)/x^(3/2) + sqrt(x + 1)/sqrt(x))/((x + 1)^2/x^2 - 2*(x + 1)/x + 1) -␣
↪1/8*log(sqrt(x + 1)/sqrt(x) + 1) + 1/8*log(sqrt(x + 1)/sqrt(x) - 1)
```

Con questo chiediamo a Sage di risolvere una equazione quadratica. Il simbolo == rappresenta l'uguaglianza su Sage.

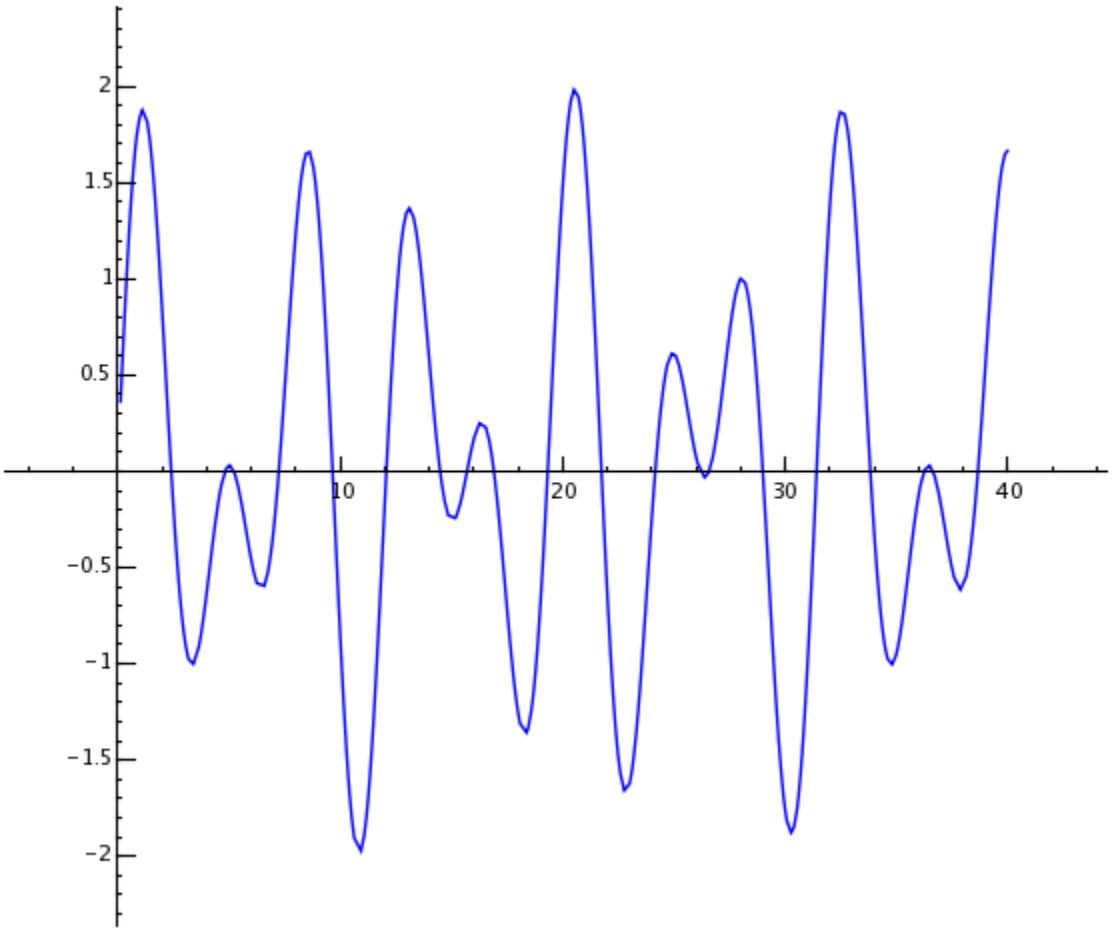
```
sage: a = var('a')
sage: S = solve(x^2 + x == a, x); S
[x == -1/2*sqrt(4*a + 1) - 1/2, x == 1/2*sqrt(4*a + 1) - 1/2]
```

```
>>> from sage.all import *
>>> a = var('a')
>>> S = solve(x**Integer(2) + x == a, x); S
[x == -1/2*sqrt(4*a + 1) - 1/2, x == 1/2*sqrt(4*a + 1) - 1/2]
```

Il risultato e" una lista di eguaglianze.

```
sage: S[0].rhs()
-1/2*sqrt(4*a + 1) - 1/2
sage: show(plot(sin(x) + sin(1.6*x), 0, 40))
```

```
>>> from sage.all import *
>>> S[Integer(0)].rhs()
-1/2*sqrt(4*a + 1) - 1/2
>>> show(plot(sin(x) + sin(RealNumber('1.6')*x), Integer(0), Integer(40)))
```


2
1.5
1
0.5
−0.5
−1
−1.5
−2
10
20
30
40

CAPITOLO 2

Power Computing con Sage

Iniziamo col creare una matrice random 500×500.

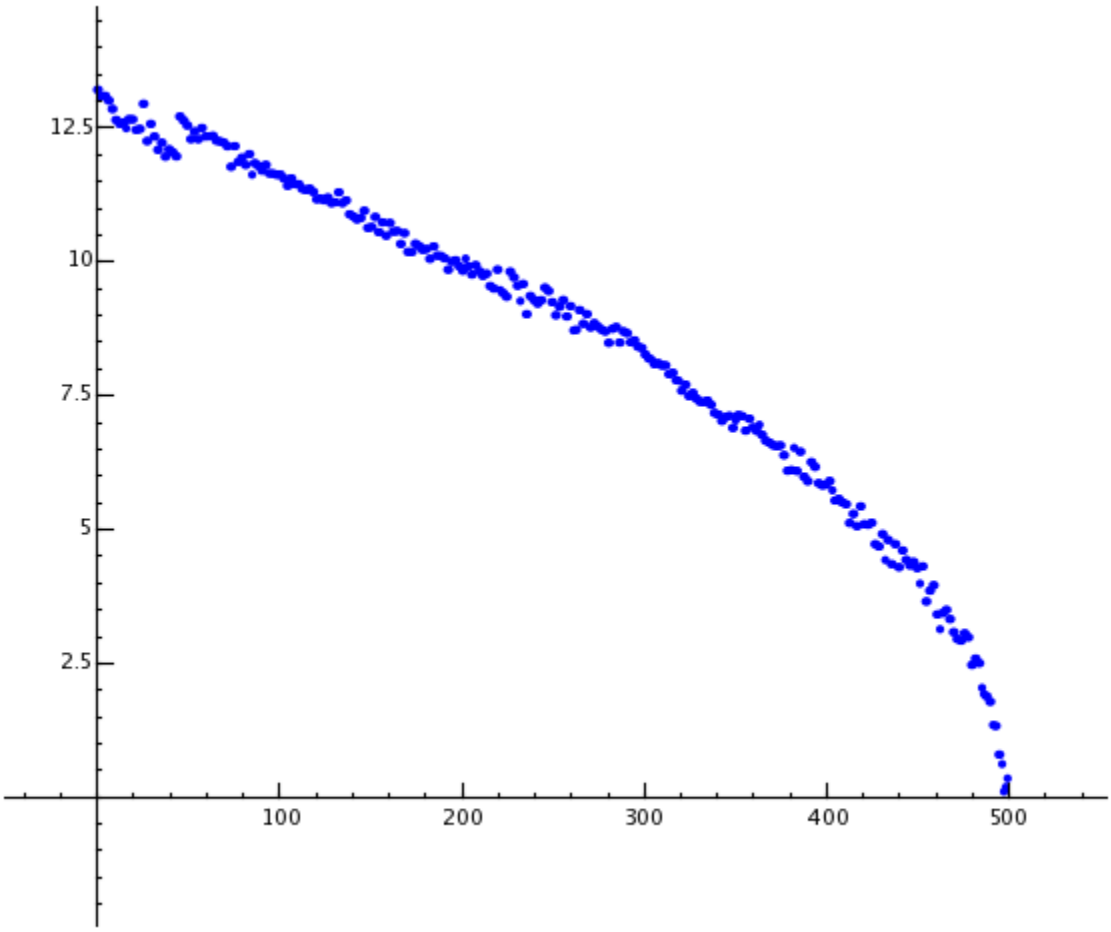
```
sage: m = random_matrix(RDF,500)
```

```
>>> from sage.all import *
>>> m = random_matrix(RDF,Integer(500))
```

Sage impiega qualche secondo per calcolare gli autovalori della matrice e farne il plot.

```
sage: e = m.eigenvalues()  #circa 2 secondi
sage: w = [(i, abs(e[i])) for i in range(len(e))]
sage: show(points(w))
```

```
>>> from sage.all import *
>>> e = m.eigenvalues()  #circa 2 secondi
>>> w = [(i, abs(e[i])) for i in range(len(e))]
>>> show(points(w))
```



Grazie alla GNU Multiprecision Library (GMP), Sage puo" maneggiare numeri molto grandi, persino numeri con milioni o miliardi di cifre.

```
sage: factorial(100)
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000000
sage: n = factorial(1000000)  #circa 2.5 secondi
```

```
>>> from sage.all import *
>>> factorial(Integer(100))
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000000
>>> n = factorial(Integer(1000000))  #circa 2.5 secondi
```

Il seguente comando mostra 100 cifre decimali di π.

```
sage: N(pi, digits=100)
3.141592653589793238462643383279502884197169399375105820974944592307816406286208998628034825342117068
```

```
>>> from sage.all import *
>>> N(pi, digits=Integer(100))
3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862
↪034825342117068
```

Questo chiede a Sage di fattorizzare un polinomio in due variabili.

```
sage: R.<x,y> = QQ[]
sage: F = factor(x^99 + y^99)
sage: F
(x + y) * (x^2 - x*y + y^2) * (x^6 - x^3*y^3 + y^6) *
(x^10 - x^9*y + x^8*y^2 - x^7*y^3 + x^6*y^4 - x^5*y^5 +
 x^4*y^6 - x^3*y^7 + x^2*y^8 - x*y^9 + y^10) *
(x^20 + x^19*y - x^17*y^3 - x^16*y^4 + x^14*y^6 + x^13*y^7 -
 x^11*y^9 - x^10*y^10 - x^9*y^11 + x^7*y^13 + x^6*y^14 -
 x^4*y^16 - x^3*y^17 + x*y^19 + y^20) * (x^60 + x^57*y^3 -
 x^51*y^9 - x^48*y^12 + x^42*y^18 + x^39*y^21 - x^33*y^27 -
 x^30*y^30 - x^27*y^33 + x^21*y^39 + x^18*y^42 - x^12*y^48 -
 x^9*y^51 + x^3*y^57 + y^60)
sage: F.expand()
x^99 + y^99
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> F = factor(x**Integer(99) + y**Integer(99))
>>> F
(x + y) * (x^2 - x*y + y^2) * (x^6 - x^3*y^3 + y^6) *
(x^10 - x^9*y + x^8*y^2 - x^7*y^3 + x^6*y^4 - x^5*y^5 +
 x^4*y^6 - x^3*y^7 + x^2*y^8 - x*y^9 + y^10) *
(x^20 + x^19*y - x^17*y^3 - x^16*y^4 + x^14*y^6 + x^13*y^7 -
 x^11*y^9 - x^10*y^10 - x^9*y^11 + x^7*y^13 + x^6*y^14 -
 x^4*y^16 - x^3*y^17 + x*y^19 + y^20) * (x^60 + x^57*y^3 -
 x^51*y^9 - x^48*y^12 + x^42*y^18 + x^39*y^21 - x^33*y^27 -
 x^30*y^30 - x^27*y^33 + x^21*y^39 + x^18*y^42 - x^12*y^48 -
 x^9*y^51 + x^3*y^57 + y^60)
>>> F.expand()
x^99 + y^99
```

Sage impiega meno di 5 secondi per calcolare in quanti modi il numero cento milioni puo" essere scritto come somma di interi positivi.

```
sage: z = Partitions(10^8).cardinality() #circa 4.5 secondi
sage: str(z)[:40]
'1760517045946249141360373894679135204009'
```

```
>>> from sage.all import *
>>> z = Partitions(Integer(10)**Integer(8)).cardinality() #circa 4.5 secondi
>>> str(z)[:Integer(40)]
'1760517045946249141360373894679135204009'
```

CAPITOLO 3

Accesso agli Algoritmi in Sage

Con Sage avete accesso ad una delle piu" grandi raccolte al mondo di algoritmi computazionali open source.